```java
//
==========================================================================
==
// PATTERN 2: LOAN APPROVAL SYSTEM (Application/Approval Workflow)
//
==========================================================================
==
// This is like Pattern 2 (Security Clearance) but for BANK LOANS
// Use this when you see: applications, approvals, status changes,
workflows
//
==========================================================================
==

//
==========================================================================
==
// FILE 1: LoanType.java (ENUM - separate file)
//
==========================================================================
==
// Right-click → New → Enum
// If SECURITY has SecurityLevel → LOAN has LoanType
// If UNIVERSITY has CourseLevel → LOAN has LoanType

enum LoanType {
    PERSONAL(1, "Personal loan for general purposes", 5000),
    AUTO(2, "Vehicle purchase loan", 50000),
    MORTGAGE(3, "Home purchase loan", 500000),
    BUSINESS(4, "Business expansion loan", 1000000);

    // EXAM TIP: Replace with hierarchy levels from YOUR exam
    // Could be: Priority, Tier, Level, Grade, etc.

    private final int level;
    private final String description;
    private final double maxAmount;

    LoanType(int level, String description, double maxAmount) {
        this.level = level;
        this.description = description;
        this.maxAmount = maxAmount;
    }

    public int level() { return level; }
    public String description() { return description; }
    public double maxAmount() { return maxAmount; }
```

```java
    // Helper method for hierarchical comparison
    public boolean requiresHigherApproval(LoanType other) {
        return this.level > other.level;
    }
}

//
==========================================================================
==
// FILE 2: CreditRating.java (ENUM - separate file)
//
==========================================================================
==
// Right-click → New → Enum
// If SECURITY has Grade → LOAN has CreditRating
// If UNIVERSITY has StudentLevel → LOAN has CreditRating

enum CreditRating {
    POOR(1, "Poor credit history", 300),
    FAIR(2, "Fair credit history", 500),
    GOOD(3, "Good credit history", 700),
    EXCELLENT(4, "Excellent credit history", 850);

    // EXAM TIP: Replace with grades/rankings from YOUR exam

    private final int code;
    private final String description;
    private final int minScore;

    CreditRating(int code, String description, int minScore) {
        this.code = code;
        this.description = description;
        this.minScore = minScore;
    }

    public int code() { return code; }
    public String description() { return description; }
    public int minScore() { return minScore; }

    // Helper method
    public boolean isBetterThan(CreditRating other) {
        return this.code > other.code;
    }
}
```

```java
//
================================================================================
==
// FILE 3: ApplicationStatus.java (ENUM - separate file)
//
================================================================================
==
// Right-click → New → Enum
// If SECURITY has Status → LOAN has ApplicationStatus
// If UNIVERSITY has EnrollmentStatus → LOAN has ApplicationStatus

enum ApplicationStatus {
    PENDING(1, "Application awaiting review"),
    UNDER_REVIEW(2, "Application being reviewed"),
    APPROVED(3, "Application approved"),
    REJECTED(4, "Application rejected"),
    CANCELLED(5, "Application cancelled by applicant");

    // EXAM TIP: Replace with workflow states from YOUR exam
    // Usually: PENDING → APPROVED/REJECTED

    private final int code;
    private final String description;

    ApplicationStatus(int code, String description) {
        this.code = code;
        this.description = description;
    }

    public int code() { return code; }
    public String description() { return description; }
}

//
================================================================================
==
// FILE 4: Applicant.java (RECORD - separate file)
//
================================================================================
==
// Right-click → New → Record
// If SECURITY has User → LOAN has Applicant
// If UNIVERSITY has Student → LOAN has Applicant

import java.util.*;

record Applicant(
```

```
    String id,
    String name,
    String email,
    CreditRating creditRating,      // Current credit rating
    double annualIncome             // Annual income
) {
    // REPLACE: Applicant with user entity from exam (Customer, Member,
etc.)
    // REPLACE: creditRating with current level/tier from exam
    // REPLACE: annualIncome with relevant metric

    public Applicant(String name, String email, CreditRating creditRating,
                     double annualIncome) {
        this(UUID.randomUUID().toString(), name, email, creditRating,
annualIncome);
    }

    public Applicant {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Applicant name required");
        }
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Valid email required");
        }
        if (annualIncome < 0) {
            throw new IllegalArgumentException("Income cannot be
negative");
        }
    }
}

//
==========================================================================
==
// FILE 5: LoanOfficer.java (RECORD - separate file)
//
==========================================================================
==
// Right-click → New → Record
// If SECURITY has Reviewer → LOAN has LoanOfficer
// If UNIVERSITY has Advisor → LOAN has LoanOfficer

record LoanOfficer(
    String id,
    String name,
    String branch,                  // Branch location
    List<LoanApplication> applications
```

```
) {
    // REPLACE: LoanOfficer with reviewer entity from exam
    // REPLACE: branch with department/division from exam

    public LoanOfficer(String name, String branch) {
        this(UUID.randomUUID().toString(), name, branch, new
ArrayList<>());
    }

    public LoanOfficer {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Officer name required");
        }
        // Make immutable (DOP Principle 2)
        applications = Collections.unmodifiableList(new
ArrayList<>(applications));
    }
}

//
==========================================================================
==
// FILE 6: LoanApplication.java (RECORD - separate file)
//
==========================================================================
==
// Right-click → New → Record
// If SECURITY has Application → LOAN has LoanApplication
// If UNIVERSITY has CourseApplication → LOAN has LoanApplication

record LoanApplication(
    String id,
    Applicant applicant,
    LoanType requestedLoanType,      // Type of loan requested
    double requestedAmount,          // Amount requested
    String purpose,                  // Purpose of loan
    ApplicationStatus status,
    LoanOfficer officer
) {
    // REPLACE: LoanApplication with application entity from exam
    // REPLACE: requestedLoanType with requested level/tier
    // REPLACE: requestedAmount with relevant amount/value

    public LoanApplication(Applicant applicant, LoanType
requestedLoanType,
                           double requestedAmount, String purpose,
                           LoanOfficer officer) {
```

```
        this(UUID.randomUUID().toString(), applicant, requestedLoanType,
             requestedAmount, purpose, ApplicationStatus.PENDING,
officer);
    }

    public LoanApplication {
        // Rule 1: Amount must be positive (REPLACE with YOUR rule)
        if (requestedAmount <= 0) {
            throw new IllegalArgumentException("Loan amount must be
positive");
        }

        // Rule 2: Amount cannot exceed max for loan type (REPLACE with
YOUR rule)
        if (requestedAmount > requestedLoanType.maxAmount()) {
            throw new IllegalArgumentException(
                "Amount exceeds maximum for " + requestedLoanType +
                " (max: " + requestedLoanType.maxAmount() + ")");
        }

        // Rule 3: Purpose must be provided (REPLACE with YOUR rule)
        if (purpose == null || purpose.isBlank()) {
            throw new IllegalArgumentException("Loan purpose required");
        }

        // Rule 4: Credit rating must be sufficient (REPLACE with YOUR
rule)
        if (requestedLoanType == LoanType.MORTGAGE &&
            applicant.creditRating().code() < CreditRating.GOOD.code()) {
            throw new IllegalArgumentException(
                "Mortgage requires GOOD or EXCELLENT credit rating");
        }
    }
}

//
==========================================================================
==
// FILE 7: LoanOperation.java (SEALED INTERFACE - separate file)
//
==========================================================================
==
// Right-click → New → Interface (then add "sealed" manually)
// If SECURITY has ApplicationOperation → LOAN has LoanOperation

// This is ONE file containing sealed interface AND all nested records
sealed interface LoanOperation<T> permits
```

```
    LoanOperation.Submit,
    LoanOperation.Review,
    LoanOperation.Approve,
    LoanOperation.Reject,
    LoanOperation.Cancel,
    LoanOperation.UpdateApplicant {

    // EXAM TIP: Add operation types based on YOUR exam workflow

    // NESTED RECORD 1: Submit operation (inside this same file!)
    record Submit(
        Applicant applicant,
        LoanType requestedLoanType,
        double requestedAmount,
        String purpose,
        LoanOfficer officer
    ) implements LoanOperation<LoanApplication> {}
    // REPLACE: Submit with initial action from exam (Create, Apply,
Request)

    // NESTED RECORD 2: Review operation (inside this same file!)
    record Review(
        LoanApplication application,
        LoanOfficer officer
    ) implements LoanOperation<LoanApplication> {}
    // REPLACE: Review with intermediate step from exam (if any)

    // NESTED RECORD 3: Approve operation (inside this same file!)
    record Approve(
        LoanApplication application,
        LoanOfficer officer,
        String approvalNotes
    ) implements LoanOperation<LoanApplication> {}
    // REPLACE: Approve with acceptance action from exam

    // NESTED RECORD 4: Reject operation (inside this same file!)
    record Reject(
        LoanApplication application,
        LoanOfficer officer,
        String rejectionReason
    ) implements LoanOperation<LoanApplication> {}
    // REPLACE: Reject with denial action from exam

    // NESTED RECORD 5: Cancel operation (inside this same file!)
    record Cancel(
        LoanApplication application,
        Applicant applicant,
```

```
        String cancellationReason
    ) implements LoanOperation<LoanApplication> {}
    // REPLACE: Cancel with withdrawal action (if in exam)

    // NESTED RECORD 6: Update applicant (inside this same file!)
    record UpdateApplicant(
        Applicant applicant,
        CreditRating newRating
    ) implements LoanOperation<Applicant> {}
    // REPLACE: UpdateApplicant with entity update from exam
}

//
================================================================================
==
// FILE 8: Result.java (SEALED INTERFACE - separate file)
//
================================================================================
==
// Right-click → New → Interface (then add "sealed" manually)

// DESIGN PATTERN: Result type for error handling
// This is ONE file containing sealed interface AND nested records
sealed interface Result<T> permits Result.Success, Result.Failure {

    // NESTED RECORD 1: Success (inside this same file!)
    record Success<T>(T value) implements Result<T> {}

    // NESTED RECORD 2: Failure (inside this same file!)
    record Failure<T>(String error) implements Result<T> {}

    // Helper methods
    static <T> Result<T> success(T value) {
        return new Success<>(value);
    }

    static <T> Result<T> failure(String error) {
        return new Failure<>(error);
    }

    // EXAM TIP: This Result pattern is OPTIONAL but shows advanced design
}

//
================================================================================
==
// FILE 9: LoanApprovalSystem.java (CLASS - separate file)
```

```
//
================================================================================
==
// Right-click → New → Class
// If SECURITY has SecurityClearanceSystem → LOAN has LoanApprovalSystem

class LoanApprovalSystem {

    // SINGLETON PATTERN: System configuration (OPTIONAL)
    private static final class Config {
        private static final Config INSTANCE = new Config();
        private final int maxApplicationsPerApplicant = 5;
        private final double minIncomeMultiplier = 3.0; // Income must be
3x loan

        private Config() {}

        public static Config getInstance() {
            return INSTANCE;
        }
    }

    // OPERATION 1: Submit loan application (FACTORY PATTERN)
    public static Result<LoanApplication> submitApplication(
            LoanOperation.Submit op) {
        try {
            // Additional business rule validation
            double minIncome = op.requestedAmount() /
Config.getInstance().minIncomeMultiplier;
            if (op.applicant().annualIncome() < minIncome) {
                return Result.failure(
                    "Insufficient income. Required: " + minIncome +
                    ", Current: " + op.applicant().annualIncome());
            }

            LoanApplication app = new LoanApplication(
                op.applicant(),
                op.requestedLoanType(),
                op.requestedAmount(),
                op.purpose(),
                op.officer()
            );

            System.out.println("Loan application submitted: " + app.id());
            return Result.success(app);

        } catch (IllegalArgumentException e) {
```

```
            return Result.failure(e.getMessage());
        }
    }

    // OPERATION 2: Move application to review
    public static Result<LoanApplication> reviewApplication(
            LoanOperation.Review op) {

        LoanApplication app = op.application();

        // Rule: Can only review PENDING applications
        if (app.status() != ApplicationStatus.PENDING) {
            return Result.failure(
                "Can only review PENDING applications. Current: " +
app.status());
        }

        // Create new application with UNDER_REVIEW status (immutability)
        LoanApplication reviewed = new LoanApplication(
            app.id(),
            app.applicant(),
            app.requestedLoanType(),
            app.requestedAmount(),
            app.purpose(),
            ApplicationStatus.UNDER_REVIEW,
            app.officer()
        );

        System.out.println("Application under review: " + reviewed.id());
        return Result.success(reviewed);
    }

    // OPERATION 3: Approve application
    public static Result<LoanApplication> approveApplication(
            LoanOperation.Approve op) {

        LoanApplication app = op.application();

        // Rule: Cannot approve already decided applications
        if (app.status() == ApplicationStatus.APPROVED ||
            app.status() == ApplicationStatus.REJECTED) {
            return Result.failure(
                "Cannot approve application with status: " +
app.status());
        }

        // Rule: Officer must match
```

```
        if (!app.officer().equals(op.officer())) {
            return Result.failure("Only assigned officer can approve");
        }

        // Create new application with APPROVED status (immutability)
        LoanApplication approved = new LoanApplication(
            app.id(),
            app.applicant(),
            app.requestedLoanType(),
            app.requestedAmount(),
            app.purpose(),
            ApplicationStatus.APPROVED,
            app.officer()
        );

        System.out.println("Application approved: " + approved.id() +
                           " - Notes: " + op.approvalNotes());
        return Result.success(approved);
    }

    // OPERATION 4: Reject application
    public static Result<LoanApplication> rejectApplication(
            LoanOperation.Reject op) {

        LoanApplication app = op.application();

        // Rule: Cannot reject already decided applications
        if (app.status() == ApplicationStatus.APPROVED ||
            app.status() == ApplicationStatus.REJECTED) {
            return Result.failure(
                "Cannot reject application with status: " + app.status());
        }

        // Create new application with REJECTED status (immutability)
        LoanApplication rejected = new LoanApplication(
            app.id(),
            app.applicant(),
            app.requestedLoanType(),
            app.requestedAmount(),
            app.purpose(),
            ApplicationStatus.REJECTED,
            app.officer()
        );

        System.out.println("Application rejected: " + rejected.id() +
                           " - Reason: " + op.rejectionReason());
        return Result.success(rejected);
```

```
}

// OPERATION 5: Cancel application (by applicant)
public static Result<LoanApplication> cancelApplication(
        LoanOperation.Cancel op) {

    LoanApplication app = op.application();

    // Rule: Cannot cancel already decided applications
    if (app.status() == ApplicationStatus.APPROVED ||
        app.status() == ApplicationStatus.REJECTED) {
        return Result.failure(
            "Cannot cancel application with status: " + app.status());
    }

    // Rule: Applicant must match
    if (!app.applicant().equals(op.applicant())) {
        return Result.failure("Only the applicant can cancel");
    }

    // Create new application with CANCELLED status (immutability)
    LoanApplication cancelled = new LoanApplication(
        app.id(),
        app.applicant(),
        app.requestedLoanType(),
        app.requestedAmount(),
        app.purpose(),
        ApplicationStatus.CANCELLED,
        app.officer()
    );

    System.out.println("Application cancelled: " + cancelled.id() +
                       " - Reason: " + op.cancellationReason());
    return Result.success(cancelled);
}

// OPERATION 6: Update applicant credit rating (after approval)
public static Result<Applicant> updateCreditRating(
        LoanOperation.UpdateApplicant op) {

    // Create new applicant with updated rating (immutability)
    Applicant updated = new Applicant(
        op.applicant().id(),
        op.applicant().name(),
        op.applicant().email(),
        op.newRating(),
        op.applicant().annualIncome()
```

```
        );

        System.out.println("Applicant credit rating updated: " +
                           updated.name() + " -> " +
updated.creditRating());
        return Result.success(updated);
    }

    // QUERY: Get applications by officer
    public static List<LoanApplication> getApplicationsByOfficer(
            LoanOfficer officer, List<LoanApplication> allApplications) {
        return allApplications.stream()
            .filter(app -> app.officer().equals(officer))
            .toList();
    }

    // QUERY: Get applications by status
    public static List<LoanApplication> getApplicationsByStatus(
            ApplicationStatus status, List<LoanApplication>
allApplications) {
        return allApplications.stream()
            .filter(app -> app.status() == status)
            .toList();
    }

    // QUERY: Check if applicant qualifies for loan type
    public static boolean checkEligibility(Applicant applicant, LoanType
loanType,
                                           double amount) {
        // Business rules
        boolean hasMinIncome = applicant.annualIncome() >=
            (amount / Config.getInstance().minIncomeMultiplier);
        boolean hasMinCredit = applicant.creditRating().code() >=
            CreditRating.FAIR.code();
        boolean withinLimit = amount <= loanType.maxAmount();

        return hasMinIncome && hasMinCredit && withinLimit;
    }
}

//
==========================================================================
==
// FILE 10: Runner.java (CLASS - separate file)
//
==========================================================================
==
```

```java
// Right-click → New → Class (check "public static void main")

class Runner {
    public static void main(String[] args) {
        System.out.println("=== Loan Approval System Demo ===\n");

        // Step 1: Create entities
        Applicant applicant = new Applicant(
            "Mary Johnson",
            "mary.johnson@email.com",
            CreditRating.GOOD,
            75000.0  // Annual income
        );

        LoanOfficer officer = new LoanOfficer("Tom Smith", "Dublin
Branch");

        // Step 2: Check eligibility
        boolean eligible = LoanApprovalSystem.checkEligibility(
            applicant, LoanType.AUTO, 30000.0);
        System.out.println("Eligible for auto loan: " + eligible + "\n");

        // Step 3: Submit application
        var submitOp = new LoanOperation.Submit(
            applicant,
            LoanType.AUTO,
            30000.0,
            "Purchase family vehicle",
            officer
        );

        Result<LoanApplication> submitResult =
            LoanApprovalSystem.submitApplication(submitOp);

        // Step 4: Pattern matching on Result (Java 17+ switch)
        switch (submitResult) {
            case Result.Success<LoanApplication> s -> {
                LoanApplication app = s.value();
                System.out.println("✓ Application submitted: " +
app.id());
                System.out.println("  Status: " + app.status());
                System.out.println("  Amount: €" + app.requestedAmount());
                System.out.println();

                // Step 5: Move to review
                var reviewOp = new LoanOperation.Review(app, officer);
                Result<LoanApplication> reviewResult =
```

```
                    LoanApprovalSystem.reviewApplication(reviewOp);

                if (reviewResult instanceof
Result.Success<LoanApplication> reviewed) {
                    System.out.println("✓ Application under review\n");

                    // Step 6: Approve application
                    var approveOp = new LoanOperation.Approve(
                        reviewed.value(),
                        officer,
                        "Good credit history and sufficient income"
                    );
                    Result<LoanApplication> approveResult =
                        LoanApprovalSystem.approveApplication(approveOp);

                    if (approveResult instanceof
Result.Success<LoanApplication> approved) {
                        System.out.println("✓ Application approved!\n");

                        // Step 7: Update applicant credit rating
                        var updateOp = new LoanOperation.UpdateApplicant(
                            applicant,
                            CreditRating.EXCELLENT  // Improved after
approval
                        );
                        Result<Applicant> updateResult =

LoanApprovalSystem.updateCreditRating(updateOp);

                        if (updateResult instanceof
Result.Success<Applicant> u) {
                            System.out.println("✓ Credit rating updated:
" +
                                u.value().creditRating());
                        }
                    }
                }
            }
            case Result.Failure<LoanApplication> f -> {
                System.out.println("✗ Application failed: " + f.error());
            }
        }

        System.out.println("\n=== Testing Invalid Application ===\n");

        // Step 8: Try invalid application (insufficient income)
        Applicant poorApplicant = new Applicant(
```

```
            "John Doe",
            "john@email.com",
            CreditRating.FAIR,
            20000.0  // Too low income
        );

        var invalidOp = new LoanOperation.Submit(
            poorApplicant,
            LoanType.MORTGAGE,  // Requires high income
            300000.0,
            "Buy home",
            officer
        );

        Result<LoanApplication> invalidResult =
            LoanApprovalSystem.submitApplication(invalidOp);

        if (invalidResult instanceof Result.Failure<LoanApplication> f) {
            System.out.println("✗ Application rejected: " + f.error());
        }
    }
}

//
==========================================================================
==
// EXAM REPLACEMENT GUIDE - LOAN vs YOUR EXAM
//
==========================================================================
==
// LOAN SYSTEM              →  YOUR EXAM SYSTEM
//
==========================================================================
==
// Applicant                →  Main user (Student, Employee, Member)
// LoanOfficer              →  Reviewer (Manager, Admin, Approver)
// LoanApplication          →  Application (Request, Submission, Proposal)
// LoanType                 →  Tier/Level (SecurityLevel, Priority, Grade)
// CreditRating             →  User level (Grade, Rank, Status)
// ApplicationStatus        →  Workflow status (State, Phase)
// Result<T>                →  Success/Failure wrapper
// LoanApprovalSystem       →  MainSystem
// LoanOperation            →  MainOperation
//
==========================================================================
==
```